From Lean Proofs to Public Claims

Release checks and trust boundaries in one formal mathematics repository

Will Cook

July 2026

Abstract

Lean verifies that a proof establishes the formal statement written in the source. It does not verify that a README or a paper describes that statement faithfully. Between a machine-checked theorem and a public mathematical claim there is a step no proof assistant performs: a person judges what the theorem means and what may responsibly be said about it. This paper describes how one public mathematics repository organises that step. A maintainer-reviewed claim record states, for selected results, the public wording, the supporting Lean declarations, the finite domain, and the conclusion that remains open. A release checker then verifies recorded relationships: declaration names near recorded source lines, exact identity of the checked Lean source, required limitation wording, and freshly rebuilt generated files. Continuous integration runs that checker and the Lean build as two separate jobs. We follow one finite result through every layer and describe a deliberate exercise in which a false strengthening of the public wording passed every check because the relevant relationship had not yet been recorded; under a proved equivalence, the altered wording amounted to claiming an open problem settled. The checks preserve reviewed relationships; they do not interpret unrestricted prose, and the mathematical judgement remains human.

1 The publication gap

The repository at https://github.com/wcook04/plectis-lean-erdos249-257 is a public Lean development around two unsolved problems in number theory, Erdős Problems 249 and 257. Both problems remain open. The project proves intermediate results, exact reformulations, and finite certificates around them; it does not claim a solution to either problem.

One theorem illustrates the problem this paper is about. Lean has checked a finite certificate at each of 28 listed scales, the largest labelled $t = 64$: at each listed scale, a finite calculation, checked in full by Lean, establishes the required property there. The registered open requirement asks for such certificates beyond every fixed cutoff. Whatever the largest checked scale is, a cutoff lies beyond it, and the finite list says nothing there. The finite list therefore does not settle the open problem.

A README can nevertheless drift, and the stakes are easy to state. By an equivalence proved in the development (Section 3), certificates beyond every fixed cutoff are not merely better evidence: their existence is equivalent to the irrationality of Problem 249 itself. An edit that presents the finite cases as completing the open requirement would, in substance, announce a solution to an open problem. Every Lean proof remains valid under that edit, because the README is not part of any proof. No program in the repository decides whether a line of English has the same meaning as a formal statement; a machine-checked theorem is, in that sense, not a self-publishing claim.

The repository keeps a *maintainer-reviewed claim record* in `docs/claims.json`. For each selected result the record states the public wording, its status, the Lean declarations that support it, the bounded domain it covers, and the registered conclusion that remains open. A

release checker, `scripts/check_release.py`, then verifies that the recorded relationships still hold across the Lean source, the record itself, the authored public pages, and the generated files. The shortest accurate summary of the division of labour is:

> Lean checks the formal proofs. A maintainer reviews what those proofs mean and how they may be described. The release machinery checks that the recorded relationships remain intact before a release.

Two boundaries should be fixed before any detail. First, the record covers *registered* claims, not every line of prose in the repository; software cannot preserve a relationship it was never pointed at, and Section 5 shows this limit operating in practice. Second, the repository does not technically force a second independent mathematician to approve a change. A single maintainer can edit a Lean statement, its claim record, and the public wording together, and every check can pass while the shared interpretation is wrong. The checks are drift detection for reviewed decisions, not a substitute for review, and they add no authority of their own.

2 The release workflow

Figure 1 shows the path in five visible parts: the Lean source; the maintainer-reviewed claim record; authored public documents; generated indexes and summaries derived from the record; and the release program and continuous-integration workflow that rerun the recorded comparisons.

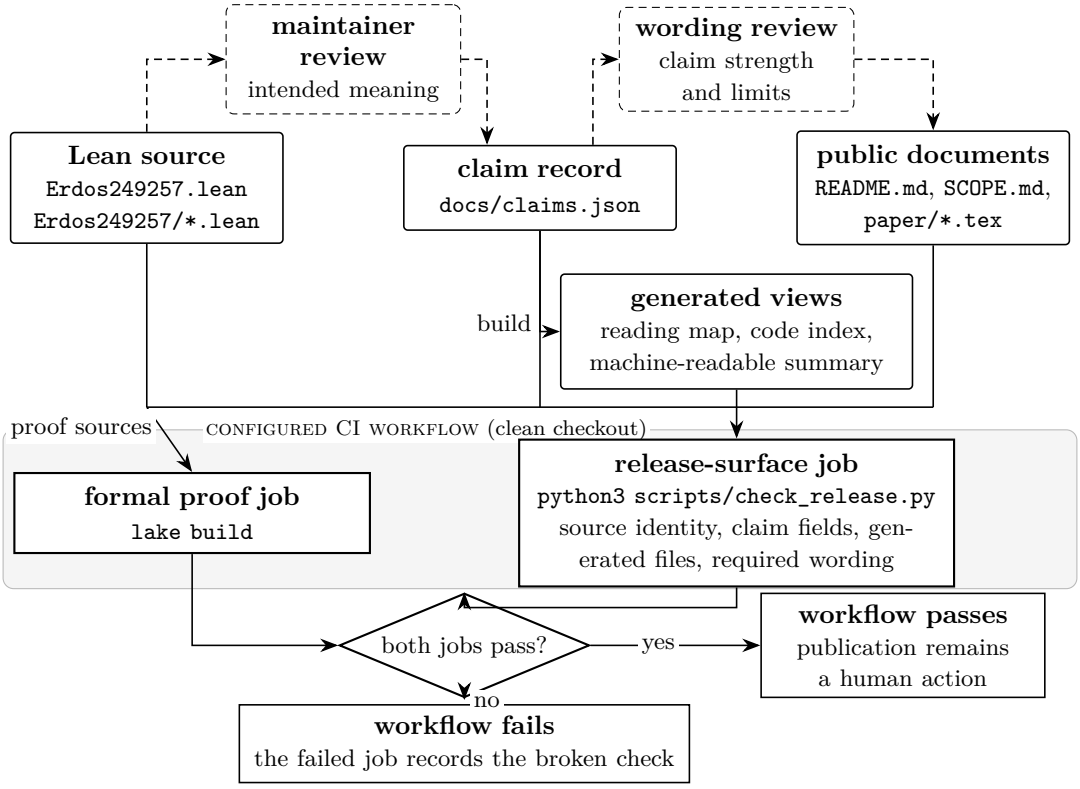

Figure 1: The configured checks for one revision. Dashed elements are human judgements; solid elements are files and programs. Three relations of different kinds meet here. Lean establishes the *proof relation*: the source proves the stated theorems. Maintainer review asserts the *semantic relation*: the recorded wording describes those theorems within stated limits. The release job checks *persistence relations*: recorded names, wording, identities, and generated files still agree. Continuous integration reruns the first and third; nothing performs the second mechanically, and neither job reads unrestricted prose or adds authority.

The division in the figure rests on a distinction the paper elaborates. Establishing that public wording is faithful to a formal theorem, within stated limits, is a semantic act: someone must read both and judge. Preserving recorded consequences of that judgement is mechanical: a program can compare names, files, and wording against a checklist. The machinery performs only the second act, answering a narrow question: once a person has approved a proof-to-claim relationship, how are later revisions kept from silently eroding it? Whether the approval was correct is a different question, and no part of the design answers it.

The two checks answer different questions. `lake build` asks whether Lean accepts the imported formal statements and proofs; it knows nothing about the README. `python3 scripts/check_release.py` asks whether the recorded relationships among proofs, record, public pages, and generated files still hold; it confirms the identity of the Lean source but does not run Lean. The workflow in `.github/workflows/lean.yml` runs them as two separate jobs on every push and pull request, in a clean checkout. A failed comparison makes `scripts/check_release.py` exit with an error, and the configured workflow treats that exit as a failing job. Blocking is a property of the repository's configuration, not of the script: the script can only refuse to succeed. The workflow's power is operational, not mathematical: a failing job can stop a configured route to release; a passing one confers no mathematical standing.

The generated views deserve demystification. They select and reorganise recorded information (a reading map, a code index, a machine-readable summary) so that a reader or a program can enter the repository without opening every file. They create no new mathematics, each names the program that builds it, and a stale view is rebuilt by that owner rather than repaired by hand. The complete file map, ownership table, and maintenance commands live in ARCHITECTURE.md; this paper keeps what the trust argument needs.

3 One claim from Lean theorem to public page

The architecture becomes concrete when one result is followed through every layer. We begin with its public meaning, not its internal name:

> Lean has checked a finite certificate at each of 28 listed scales through $t = 64$. The registered open requirement asks for certificates beyond every fixed cutoff. The finite list does not settle it.

3.1 What the certificates are for

The result belongs to Erdős Problem 249, which asks whether the totient constant

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is irrational, where $\varphi(n)$ counts the integers in $\{1, \ldots, n\}$ sharing no factor with n. The development proves an exact reformulation: S is irrational precisely when a family of finite arithmetic statements holds at every parameter; a one-parameter form runs along the scales $H_t = \operatorname{lcm}(1, \ldots, t)$. A *certificate* at a given scale is a finite, fully checkable computation witnessing the statement there: a certain finite weighted difference of totient values leaves a residue, modulo a power of two, outside a specified neighbourhood of zero. The project calls such a kernel-checked certificate a *certified kill*: each one eliminates the family of candidate rational representations of S tied to its parameters. Certificates at scales beyond every fixed cutoff would eliminate every candidate, and by the reformulation their existence is equivalent to the irrationality itself. Certificates on a finite list, however long, are not.

Writing $\mathrm{Cert}(t)$ for "a certificate exists at scale t", the two statements then read:

$$\begin{aligned} \text{checked:}\quad & \mathrm{Cert}(t) \text{ for every } t \text{ in a fixed list of 28 scales;} \\ \text{open:}\quad & \text{for every cutoff } T,\ \mathrm{Cert}(t) \text{ for some } t > T. \end{aligned}$$

The boundary between them is a matter of logical form, not of scale. No finite list, however long or expensive to verify, implies the second statement without a further theorem that produces new witnesses. Thus $t = 64$ is the endpoint of this verified computation, not a frontier that Lean has proved will continue. This is the boundary an edit can silently delete.

3.2 The formal evidence

The Lean module `Erdos249257/DiagonalPincerCertificatesT64.lean` defines the explicit list of scales,

$$\{1, 2, 3, 4, 5, 7, 8, 9, 11, 13, 16, 17, 19, 23, 25, 27, 29, 31, 32, 37, 41, 43, 47, 49, 53, 59, 61, 64\},$$

the scales up to 64 at which the diagonal H_t attains a new value, and proves the assembling theorem `certifiedKill_diagonal_all_imported_through_t64`: for every listed t, the theorem gives a finite checking depth d_t and a decidable arithmetic witness at (H_t, H_t, d_t). Imported modules prove the cases; the terminal theorem combines them. The proofs use kernel-checked arithmetic, including the tactic `decide`, which evaluates a decidable statement inside Lean's proof kernel. The root `Erdos249257.lean` imports the module, so `lake build` checks it.

3.3 The reviewed record

Table 1 shows the substance of the corresponding entry in `docs/claims.json`, ordered as a reader meets it: what is claimed, where it stops, what remains open, then the supporting machinery. The entry also records an exact source line for each declaration; those coordinates are omitted from the table.

Field	Content
Public statement	"Kernel-checked certified kills for small periods and at 28 explicit lcm-diagonal scales through $t = 64$, plus finite certificate shards, each discharged by `decide`."
Bounded domain	The listed small periods, the 28 scales above, and the parameters encoded by the finite certificate shards.
Remaining open	A link to the registered open requirement *unbounded certificate supply*: produce certified witnesses at unbounded parameters.
Status	*verified finite instance*: Lean checked stated finite inputs; this is not a score.
Supporting declarations	Five named Lean declarations with recorded modules and source lines, ending in `certifiedKill_diagonal_all_imported_through_t64`.
Claim identifier	`certified_kill_instances`

Table 1: The reviewed claim entry for the worked example, exact source coordinates omitted. The pairing of the positive statement with its bounded domain and its remaining-open link is the point: public overstatement usually happens by deleting a boundary, not by inventing a theorem.

The public statement is exact but not self-explanatory. Two of its terms appear in Section 3.1, *certified kills* and the *lcm-diagonal scales* H_t; the third, *certificate shards*, names smaller reusable certificates at fixed parameters, each discharged by the kernel tactic `decide`. A record can be precise without being pedagogy: its job is to be the stable object that explanatory prose, the release checker, and the generated views refer to; the explanation a cold reader needs is the job of prose like this section.

The fields differ in kind, and the difference matters for what "reviewed" means. The public statement, bounded domain, remaining-open link, and status are semantic fields: they record the outcome of a human judgement, and only a person can meaningfully change them; the status vocabulary is fixed in the record, and the checker rejects a status outside it. The claim identifier is a lookup handle for software, which is why it appears last rather than in place of the wording, and the per-declaration source lines are bookkeeping that tooling may refresh when code moves; refreshing them re-establishes nothing semantic. What no field can do is make the English faithful to the Lean: the maintainer judged that the public statement above describes the five declarations correctly, and the record stores the outcome of that judgement, not a justification of it. The judgement itself is mundane: read the terminal theorem, check that the wording claims the listed scales and no more, confirm that the unbounded requirement stays named as open. What that decision must survive is not a rival mathematician but an ordinary rewrite months later.

3.4 What the checker verifies here

For this claim, `scripts/check_release.py` verifies a finite, listed set of relationships. Each declaration name must appear in the Lean source within three lines of its recorded coordinate; this is a drift check on names and positions, not a check that the declarations entail the public wording. The checked Lean source itself must be exactly the formal-source checkpoint named in the claim record: the checker compares the supported root and the whole library directory against that recorded version and rejects the release if they differ or if untracked Lean files are present, so the verified names cannot refer to an older source than the one being shipped. The required limitation wording must remain present on the registered public pages. The generated views must equal a fresh rebuild from their sources. Presence, identity, and proximity are all the checker can test. Lean checks the theorem; the checker checks the recorded description around it; a person remains responsible for the claim that the two say the same thing.

4 What the checks establish

Four kinds of assurance meet in Figure 1, and informal talk about "verification" blends them. Lean provides *proof* assurance: the written formal statements follow from their imported environment. Maintainer review provides *interpretive* assurance: a recorded judgement that selected wording describes selected theorems within stated limits. The release checker provides *persistence* assurance: the recorded relationships still hold at this revision. Continuous integration provides *execution* assurance: the configured jobs ran on the named revision and passed. These are not interchangeable, and success at one level does not flow upward. A green run establishes a conjunction: Lean accepted the proofs, and every recorded relationship held. It does not establish that the wording is faithful, nor that every consequential claim was registered. Table 2 states the guarantee and non-guarantee of each layer, because combining layers in prose makes the guarantee sound stronger than it is.

Parts of the release checker are stronger than the phrase "consistency check" suggests, and worth stating. As a project-source policy, the checker rejects `sorry`, `admit`, new `axiom` declarations, and `native_decide`; the last exclusion keeps finite certificates on the repository's kernel-checkable computation path. This lexical policy does not show that imported libraries use no classical axioms, and the pinned toolchain and imported definitions remain part of the trust base. The checker also compares the entire public proof surface byte for byte against the recorded formal-source checkpoint and rejects untracked Lean files. It rebuilds the generated views and requires exact agreement rather than plausible freshness, verifies the recorded source and PDF fingerprints of the shipped papers, and carries its own negative examples: deliberately wrong inputs that must keep failing, so that a check cannot silently become vacuous. None of

Layer	What it can establish	What it cannot establish
Lean build and kernel	The written formal statements are proved from their imported definitions and assumptions, in the pinned toolchain.	That a formal statement is the one the author intended, or that any English description of it is faithful.
Maintainer review	A recorded judgement that a formal statement has the intended meaning and that selected public wording describes it within stated limits.	That Lean accepts the proofs; that every important claim was noticed and registered; its own independence or quality.
Release checker	That recorded names, coordinates, fields, links, required wording, source identity, and generated files agree with the declared rules, and that its own deliberately wrong examples still fail.	The meaning of unregistered prose; the completeness of the recorded checklist; the correctness of the human judgements it preserves.
Continuous integration	That both jobs ran in a clean checkout of the pushed revision and exited successfully.	Independent mathematical approval; anything beyond what the two jobs themselves establish.

Table 2: The trust boundary, one layer at a time.

this reaches semantics. A fingerprint can prove two files identical; it cannot say which of two differing files expresses the right mathematics.

When a file changes, the revision passes through the same stations: proof validation with `lake build`; semantic reconsideration, a mathematical judgement about whether statements, assumptions, or intended meaning moved; record and page updates where public meaning changed, under the review rules in `docs/methodology.json`; regeneration of derived views by their named builders; and structural comparison with `python3 scripts/check_release.py`, after which the push repeats both jobs in a clean checkout. A prose-only edit skips only the first station: if the words now make a stronger or different mathematical claim, the record and the review must change with them. The command list is maintenance detail in the repository guide.

5 A boundary the checklist missed

The limits in Table 2 are not hypothetical. They were probed by a deliberate exercise, recorded in `docs/publication_evidence.json`, whose shape is worth describing exactly.

The retained maintainer record reports that, at a pinned repository checkpoint, ten deliberate false edits were applied to an isolated working copy, one at a time, with the copy restored between runs and the full release gate run after each edit. The edits targeted the seven recorded relationship families: status labels, record structure, source coordinates, paper link anchors, boundary wording, generated-view freshness, and size budgets. Representative edits upgraded a conditional status to "proved here", deleted an open-boundary clause from the README, drifted a declaration coordinate by forty lines, introduced `native_decide` into a proof, and hand-edited a generated module count by one.

According to that record, nine of the ten edits were rejected. One escaped: an edit strengthening a finite-evidence boundary clause in the README, replacing wording that the finite cases *do not supply* the open requirement with wording that they *complete* it. Under the registered equivalence of Section 3.1, the altered page claimed, in effect, that the open problem was settled. The formal proofs were untouched, and the release gate passed in full, because that clause was not among the recorded relationships. The error was outside the written checklist, and the checker preserved exactly what it had been given.

One escape carries more information than nine rejections: it refutes, by counterexample, the strongest reading a green run invites, that passing every configured check certifies faithful public prose. The refutation needs no statistics, and the exercise supplies none.

The repair had two parts. The clause became a declared anchor: required wording on the public page, recorded in the checklist. And the checklist derives from each such anchor an opposing example: a copy of the page with the clause weakened or removed must be rejected. The intact text was then rechecked and passed; the same false edit now fails. What the repaired configuration demonstrates is narrow: for this one relationship the check is not vacuous, because a known disagreeing example fails; that is coverage growth, not general effectiveness. The other nine edits were not rerun against the extended checklist, and no aggregate result is claimed for the repaired configuration.

This exercise demonstrates a coverage boundary, not a reliability score. Its ceilings should be stated as plainly as its outcome. The edits were authored by the checker's author, and most relationship families were probed by exactly one edit. The exercise covers one repository, one maintainer, and one working style, with no comparison against ordinary continuous integration or careful manual review. The original run logs were not retained; the evidence file records the protocol, the outcomes, and these limitations, and a deterministic reconstruction of the edits can be replayed against the current checker, but the reconstruction is not the original evidence. What survives all of these ceilings is the design lesson:

> The program can preserve a relationship only after a person has identified and recorded that relationship.

The residual risk follows the same line: requiring a clause to be present does not detect a contradictory stronger claim elsewhere on the page. Coverage grows only by the route above, notice a relationship, record it, give it an example that must fail, and the exercise is one full turn of that loop.

6 Scope, reuse, and limits

The failures the design addresses. The checks are aimed at accidental, single-point drift after a sound review: a declaration moves and its recorded coordinate goes stale; a generated view is hand-edited or stale; a required limitation clause disappears in a rewrite; a status is quietly upgraded; the shipped Lean tree ceases to be the reviewed one; a check decays into one that can no longer fail. Each changes one endpoint of a recorded relationship while the others stand still; mechanical comparison suits that shape of failure.

The failures it does not address. Three families remain outside by design. First, semantically equivalent damage: a paraphrase that strengthens a claim without touching a registered anchor, a contradictory assertion elsewhere on the page, misleading emphasis, or a new public document the record never mentions; requiring a clause to be present is not the same as requiring the page to agree with it. Second, coordinated change: a maintainer with authority over source, record, and prose can move all three together, and perfect structural agreement then preserves a shared error; what remains is the quality and governance of the review itself. Third, wrong review: if the original judgement was mistaken, persistence faithfully preserves the mistake. The machinery has no opinion about the judgements it protects.

Essential and contingent limits. Some limits are essential to the chosen shape: while claims live in unrestricted prose beside an external record, human interpretation cannot be eliminated, coverage is selective (registration is itself a judgement, weighted here toward headline results and wording whose accidental strengthening would change the repository's public status),

and unregistered wording is invisible. Others are contingent and could be engineered away without changing the architecture: the single maintainer, the three-line proximity window, exact-wording anchors, one edit per relationship family in the exercise, and the unretained logs.

Reuse. Another formal project could reuse the pattern without copying any file format. The portable obligations are short to state: a defined unit of public claim; each claim naming its formal evidence and the source state that evidence lives in; explicit scope, assumptions, or finite domain, with consequential non-conclusions recordable; a human review act distinguishable from machine validation; named owners for derived public views; proof checking and publication checking as separate checks, each able to fail the configured workflow; a deliberately disagreeing example for every invariant that matters; and coverage represented as selective rather than assumed complete. Stronger variants lie beyond this minimum, each buying coverage at a cost: generating the public wording from the record instead of anchoring authored prose; extracting statements and assumptions from the proof assistant instead of recording source lines; requiring independent review for designated claims; constraining critical claims to a controlled vocabulary; or adding advisory semantic tooling that proposes, but never certifies, candidate discrepancies. This repository occupies that family's conservative end.

What is not established. The architecture does not establish a solution to either Erdős problem; that a formal statement is the statement the author intended; that software understood any unregistered prose; that the record is complete; that one maintainer's review is independent or adequate; or that the design transfers to other projects with its behaviour intact. A large number of passing comparisons measures none of those things. What a green workflow run establishes is narrower: Lean accepted the formal proofs, and every configured comparison passed for the named revision. This paper is an architecture note with one bounded case study, not an empirical evaluation of the design.

7 Related systems

Related systems sit on two axes: where in the life of a formalisation they act, and where their semantic structure lives. Proof blueprints act before and during formalisation, connecting plans to Lean declarations and tracking dependency and completion: `leanblueprint` for authored blueprints [1], LeanArchitect for generating and maintaining them [2]. The claim record here acts after: not "what remains to formalise", but "what may the finished formalisation be said to establish, and with what limits". Lean Atlas supports judging whether a formal statement expresses the intended mathematics [3]; this system takes that judgement as input and preserves its recorded consequences. On the second axis, Isabelle/DOF places typed, checked structure inside formal documents themselves [4], gaining coverage over whatever is written in that structure at the cost of constraining the document; this repository leaves prose unrestricted and checks an external record, buying a clear human review surface at the cost of open-world coverage. Documentation-consistency systems such as CASCADE interpret prose to generate tests [5], reaching text that explicit records cannot see at the price of probabilistic interpretation; the design here takes the opposite point and leaves unrecorded drift invisible. The deliberately wrong copies of Section 5 are known-answer regression fixtures in the mutation-analysis tradition [6, 7], keeping checks non-vacuous rather than computing a mutation score. Outside formal mathematics, the claim record is a lightweight relative of requirements traceability [8] and of assurance cases [9]: a claim with named evidence, explicit scope, and recorded non-conclusions is close to a claim–evidence–warrant structure, except that the warrant here is an unformalised human judgement and the known defeaters, unregistered prose and reviewer error, are stated rather than argued away. What formal mathematics adds to that older picture is an unusually

strong evidence layer beside an unusually mutable public one; the distance between them is the subject of this paper.

8 Conclusion

The paper opened with a distance: 28 checked scales on one side, certificates beyond every fixed cutoff on the other, and a proved equivalence making the far side the open problem itself. Everything afterwards answers one question about that distance: what would it take for the words that mark it, the listed scales, the finite domain, the registered open requirement, to survive years of edits by the hands free to change them?

The answer is narrow and exact within its bounds. The workflow does not make prose formally true. It makes a selected set of proof-to-prose relationships explicit, reviewed, and mechanically persistent, and it makes the configured workflow fail when one of them breaks. The worked example shows both halves: the repaired checker accepts the intact wording and rejects the reconstructed false edit, while the earlier unrecorded boundary passed unnoticed. Three achievements should stay separate, because none implies the others: a theorem is proved; its public interpretation is reviewed; the reviewed relationship is preserved. Lean supplies the first. Only a person can supply the second. The machinery here supplies the third, for the relationships it has been given.

The evidence here should be weighed at three strengths. That the architecture exists and behaves as described at one revision is demonstrated. That reviewed proof-to-prose relationships can be made mechanically persistent, without pretending that software interpreted the prose, is argued, and the case study supports it. That such an architecture reduces consequential publication drift at acceptable maintenance cost across projects is a hypothesis for future evaluation, not a result of this paper. Both Erdős problems remain open, and nothing here changes their status: the unbounded certificate supply is exactly as unproved after these checks as before them.

A Reproducibility

The reviewed claim record is `docs/claims.json`. Its release block names the saved Git revision of the Lean source, and the release checker requires an exact match. The paper omits that changing identifier because the record is its single owner; a copy in prose would be a second value that can drift.

The paper inventory is `docs/publication_contract.json`: source and rendered fingerprints for each shipped manuscript, with validation commands. The record of the Section 5 exercise is `docs/publication_evidence.json`: protocol, outcomes, and limitations. Its deterministic reconstruction is described by `experiments/publication_mutations.json`; the replay command is `scripts/run_publication_mutations.py` with `--verify-operators`. The original raw outputs of the historical runs were not retained, and the reconstruction does not claim to be them.

The papers build with Tectonic or standard LaTeX via `make -C paper`. Buildability, not byte-identical output across TeX engines, is the claim. The shipped PDF is the tracked root copy, whose fingerprint the paper inventory records and checks. These identities establish which artefacts are named, not that the artefacts are interpreted correctly; the appendix is one more instance of the paper's subject.

References

[1] P. Massot, *leanblueprint: a plasTeX plugin for formalization blueprints*, project repository, accessed July 2026.

[2] T. Zhu, P. Monticone, J. Avigad, and S. Welleck, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, 2026, arXiv:2601.22554.

[3] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv:2604.16347.

[4] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–292.

[5] T. Kiecker et al., *CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation*, 2026, arXiv:2604.19400.

[6] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, IEEE Computer 11(4), 1978, pp. 34–41.

[7] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678.

[8] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, in *Proc. IEEE International Conference on Requirements Engineering*, 1994, pp. 94–101.

[9] T. Kelly and R. Weaver, *The Goal Structuring Notation: a safety argument notation*, in *Proc. DSN Workshop on Assurance Cases*, 2004.